



Mobile Application Development
(Theory)
(Assignment : 01)

Submitted By:

Name: Areeba Shehzadi

Class : BSCS – 5A

Reg No : 242201007

Submitted To:

Sir Uzair

Submission Date / Day:

5th October 2026 / Monday

Publishing an Android App to Google Play: Versioning, Signing and Release Workflow

Contents

1	Introduction	2
2	Part 1: APK Versioning	2
3	Part 2: Generating a Signed Build	3
4	Part 3: Publishing to Google Play	6
5	Practical Task	6
6	Play Console Publishing Plan and Current Status	8
7	Conclusion	9

1 Introduction

Publishing an Android app involves more than writing code. The app needs a proper version number so that updates can be tracked, it must be digitally signed so that its author can be verified, and it must be uploaded to the Google Play Console together with a store listing, a privacy policy and several declarations. This report explains each of these stages, answers the ten understanding questions, and documents a practical demonstration in which a basic “Hello World” app was given a version, signed, and exported as an Android App Bundle (AAB).

2 Part 1: APK Versioning

Q1. What are `versionCode` and `versionName`, and why are they important?

`versionCode` is a whole number that Android and Google Play use internally to decide which build of an app is newer. `versionName` is a text label, such as “1.0” or “2.3.1”, that is shown to users and developers. Users never see the `versionCode`, but the store depends on it to manage updates, while the `versionName` helps people understand which release they have installed.

Q2. What happens if you do not increase the `versionCode` when updating your app?

Google Play Console rejects an upload whose `versionCode` is the same as, or lower than, a previous one. Even if the file were installed manually, the device would not treat it as an upgrade of the existing app. Every new release must therefore carry a higher `versionCode`.

Q3. Where can you find and edit these values in Android Studio?

Both values are in the module-level `build.gradle` file (Gradle Scripts → `build.gradle` (Module :app)) inside the `defaultConfig` block. They can also be viewed and edited through the Project Structure dialog (Ctrl+Alt+Shift+S). After editing, the project must be synced with Gradle.

3 Part 2: Generating a Signed Build

Q4. What is a `.jks` (keystore) file, and why is it critical?

A Java KeyStore (`.jks`) file stores the private key used to sign an app. The signature proves that the app and all its future updates come from the same developer. If the key is lost, the developer may not be able to publish updates under the same app identity, and if it is stolen, someone else could sign apps pretending to be the developer.

Q5. What is the difference between `.apk` and `.aab` files?

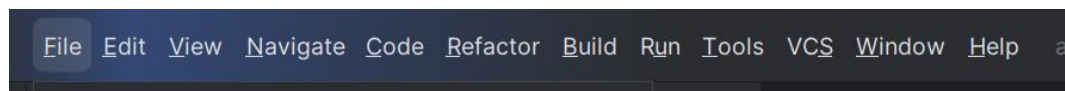
An APK is a complete installable package that can be installed directly on a device. An AAB is a publishing format that is uploaded to Google Play, which then generates optimized APKs for each device type, for example matching screen density and CPU architecture. This usually results in smaller downloads for users. Google Play requires new apps to be published as AAB files.

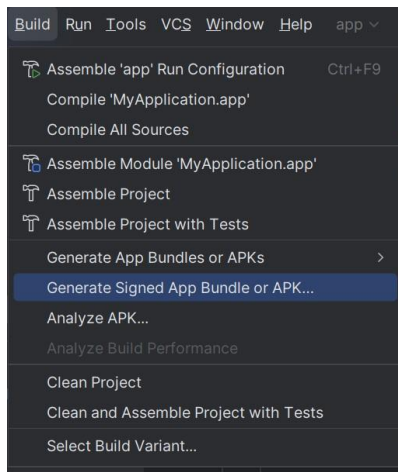
Q6. Step-by-step: how to generate a signed release file in Android Studio

1. Open the project and set `versionCode` and `versionName` in `build.gradle`, then sync Gradle.

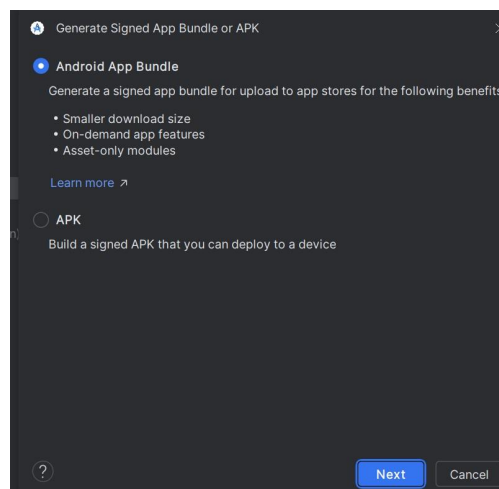
```
defaultConfig {  
    applicationId "com.areeba.hellorelease"  
    minSdk 24  
    targetSdk 37  
    versionCode 1  
    versionName "1.0"
```

2. Choose Build → Generate Signed App Bundle / APK.

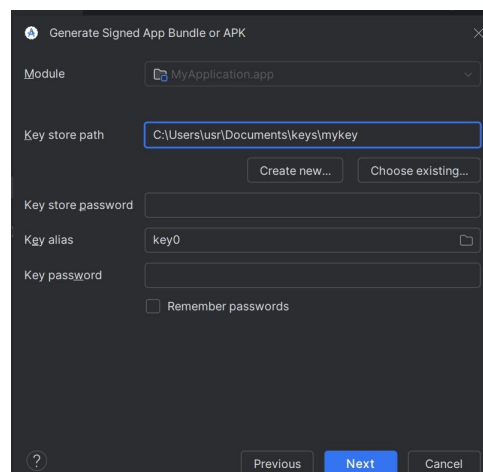




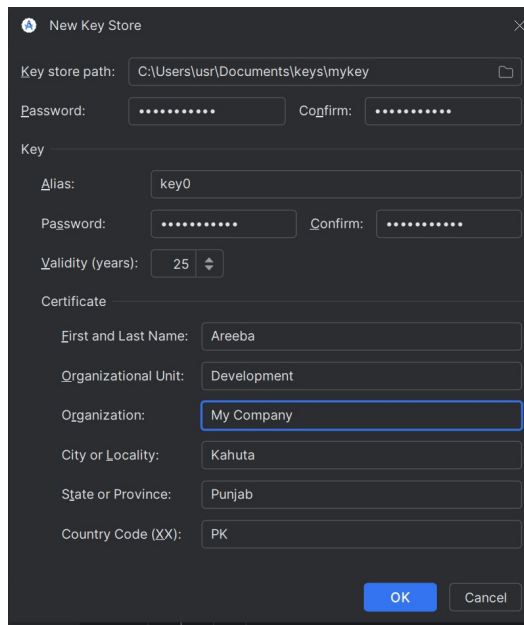
3. Select Android App Bundle and click Next.



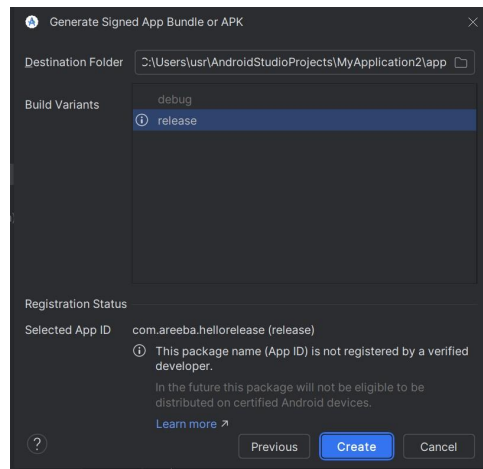
4. Click Create new... next to the key store path.



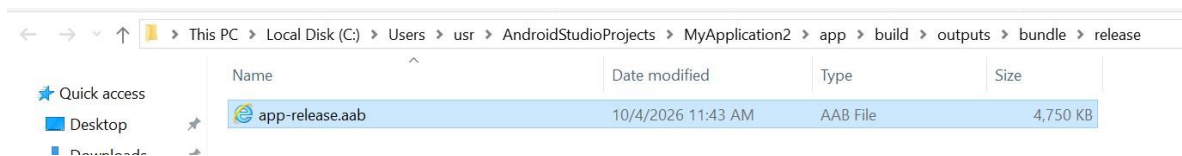
5. Enter the keystore location and file name, a keystore password, a key alias, a key password, a validity of 25 years, and at least the developer's name in the certificate section. Click OK.



6. Click Next, select the release build variant, and click Create/Finish.



7. Wait for Gradle to finish. A notification confirms success, and the file `app-release.aab` is created under `app/build/outputs/bundle/release/`.



Q7. What precautions should you take with your keystore file and passwords?

The keystore and its passwords should be backed up in at least two safe places, such as cloud storage and an external drive. They should never be uploaded to a public repository like GitHub or shared with other people. Passwords should be stored in a password manager or another secure place. Developers can also enrol in Play App Signing, where Google holds the final app signing key, so a lost upload key can be reset.

4 Part 3: Publishing to Google Play

Q8. What is required to open a Google Play Developer account?

A Google account is needed, along with a one-time registration fee of \$25 paid by card, acceptance of the Developer Distribution Agreement, and identity verification using a government-issued ID, proof of address and phone verification. Personal accounts created after 13 November 2023 must also complete a closed test with at least 12 testers for 14 continuous days before they can apply for production access. Organization accounts are treated differently and need business details.

Q9. Major steps in publishing an app on the Google Play Console

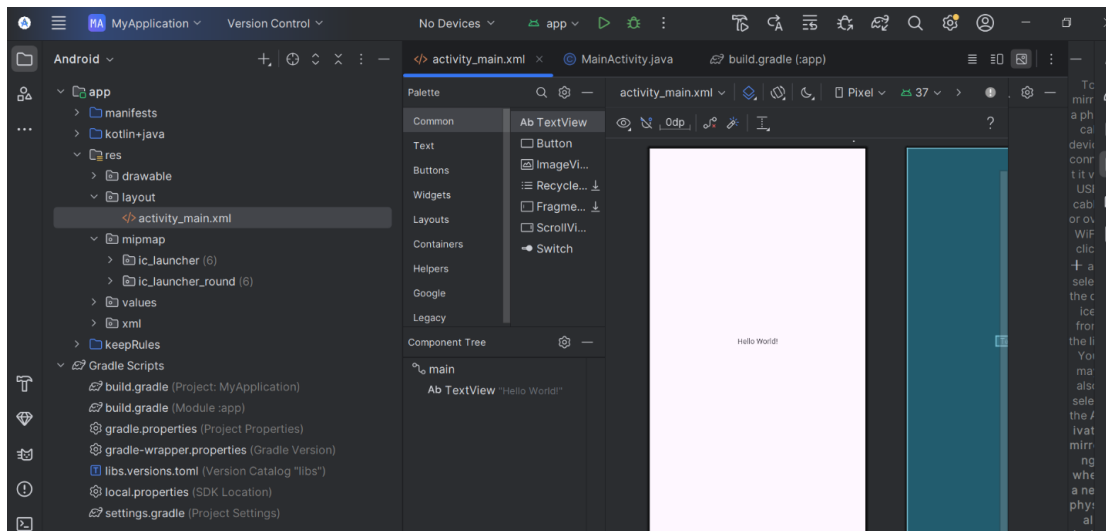
1. Create a developer account and complete verification.
2. Click Create app and enter the app name, language, app or game type, and free or paid status.
3. Complete the dashboard tasks: privacy policy, app access, ads declaration, content-rating questionnaire, target audience, and data safety form.
4. Prepare the store listing: title, short and full description, app icon, feature graphic and screenshots.
5. Create a release on a testing track (internal or closed) and upload the signed .aab file.
6. For new personal accounts, run the closed test with 12 testers for 14 days, then apply for production access.
7. Create a production release, submit it for review, and roll it out once approved.

Q10. Common errors or rejections and how to avoid them

- **Duplicate or low `versionCode`:** always increase it for every upload.
- **Using `com.example` as the package name:** choose a unique application ID before the first upload.
- **Missing or invalid privacy policy:** host a public policy page and add its link.
- **Outdated target API level:** Google requires new apps and updates to target a recent Android version (API 36 from 31 August 2026), so check `targetSdk`.
- **Uploading an APK instead of an AAB:** export a signed AAB.
- **Inaccurate declarations or misleading descriptions:** fill in the data safety form and content rating honestly and read the Play policies before submitting.
- **Not meeting the testing requirement:** recruit enough testers early and ask them to actually use the app during the 14 days.

5 Practical Task

A basic “Hello World” application named MyApplication was created in Android Studio with the application ID `com.areeba.hellorelease`. The version was set to `versionCode 1` and `versionName “1.0”`, with `minSdk 24` and `targetSdk 37`. A signed release bundle was then generated.



Step 1: Version settings

Figure 1 shows the defaultConfig block in build.gradle.

```

10
11     defaultConfig {
12         applicationId "com.areeba.hellorelease"
13         minSdk 24
14         targetSdk 37
15         versionCode 1
16         versionName "1.0"
17
18         testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
19     }

```

Figure 1: versionCode and versionName in build.gradle

Step 2: Generate Signed Bundle

Figure 2 shows the Generate Signed Bundle window with the release build variant selected.

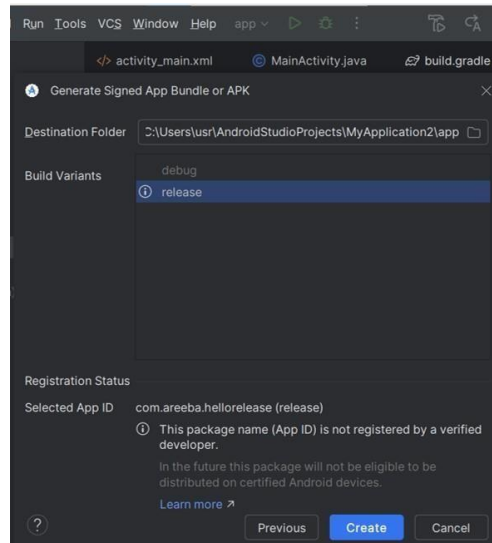


Figure 2: Generate Signed App Bundle window

Step 3: Resulting AAB file

Figure 3 shows the generated `app-release.aab` file (about 4.63 MB) in the output folder.

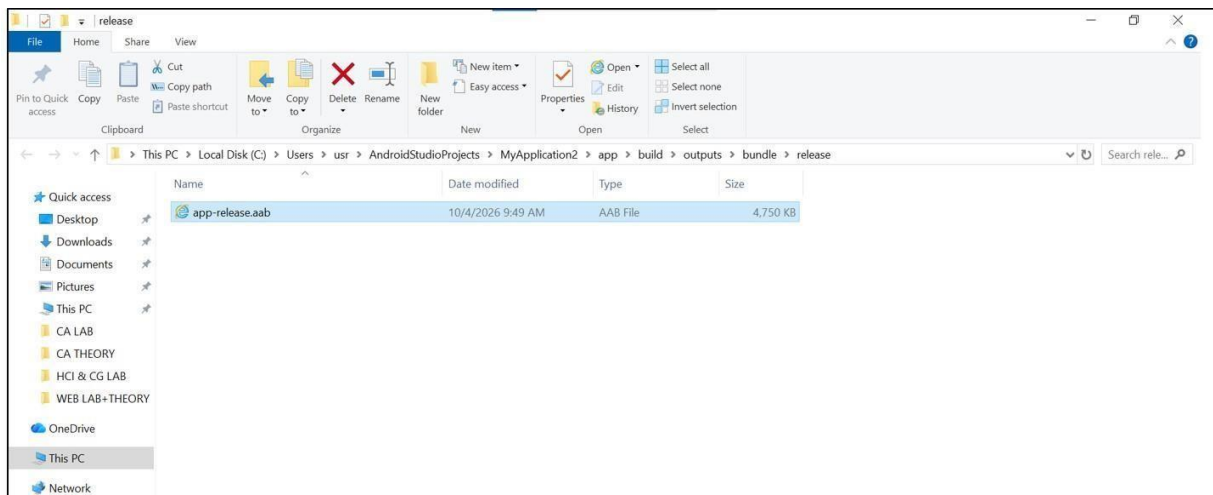


Figure 3: Generated `app-release.aab` file

6 Play Console Publishing Plan and Current Status

The signed Android App Bundle (`app-release.aab`) is ready for upload. The remaining publishing workflow in the Google Play Console is summarised below.

- **Ready:** application ID `com.areeba.hellorelease`, versionCode 1, versionName “1.0”,

target SDK 37, and a signed release AAB.

- **Next:** create a personal developer account (one-time \$25 fee) and complete identity verification.
- **Then:** create the app, complete the store listing, privacy policy, content rating, target audience and data safety sections.
- **Testing:** upload the AAB to a closed testing track and keep at least 12 testers opted in for 14 continuous days, as required for new personal accounts.
- **Release:** apply for production access, then roll out the release after Google's review.

Because of the mandatory 14-day closed testing period, a public production release cannot be completed immediately after the first upload.

7 Conclusion

This assignment covered the full path from a finished app to a store-ready release. Version codes make updates traceable, the keystore proves the developer's identity, and the AAB format lets Google Play deliver smaller, device-specific installs. The Play Console adds further requirements such as a privacy policy, declarations, identity verification and, for new personal accounts, a 14-day closed test. Understanding and preparing for these steps in advance helps avoid common rejections and delays.